

Introduction à la programmation Python

Support de cours complet — IUT 1re année

Séance 0

Présentation du cours

Objectifs, programme et modalités

Ce cours vous donnera les bases nécessaires pour **concevoir des programmes Python appliqués aux données et à la mesure**.



À l'issue de cette introduction, vous connaîtrez l'organisation du cours, les compétences visées et les modalités d'évaluation.

Organisation du module

	Temps	Contenu
16 h		cours et activités pratiques, répartis en 4 séances
2 h		examen final

Les quatre séances suivent une progression : des premières instructions Python jusqu'à l'acquisition de mesures provenant d'un instrument.

La séance 0 présente le cadre du module. Les 16 heures annoncées correspondent aux quatre séances d'apprentissage décrites dans le programme ; l'examen final de 2 heures s'y ajoute.

Objectifs du cours

À la fin du module, vous devrez être capables de :

- connaître les bases du langage Python ;
- concevoir des programmes en utilisant les outils du langage ;
- manipuler des valeurs numériques, du texte et des fichiers ;
- communiquer avec des instruments de mesure.

Modalités d'évaluation

Évaluation	Part de la note finale
QCM en début de séance	25 %
Examen final de 2 h	75 %

Les QCM vérifient régulièrement l'acquisition des notions essentielles. L'examen final évalue votre capacité à mobiliser l'ensemble du cours.

La note obtenue aux QCM représente globalement 25 % de la note du module. Il est donc utile de revoir les notions de la séance précédente avant chaque nouveau cours.

Programme — Séances 1 et 2

Séance 1 — Premiers programmes Python

Écrire un script simple, manipuler des variables et des types, effectuer des calculs, saisir et afficher des valeurs, puis prendre des décisions avec des conditions.

Séance 2 — Organiser et répéter les traitements

Structurer un programme avec des fonctions, utiliser les méthodes d'objets existants, manipuler des listes et du texte, puis automatiser des traitements à l'aide de boucles.

Programme — Séances 3 et 4

Séance 3 — Exploiter des données scientifiques

Lire et écrire des fichiers, traiter des données CSV et utiliser NumPy pour réaliser des calculs sur des séries de valeurs.

Séance 4 — Visualiser et acquérir des mesures

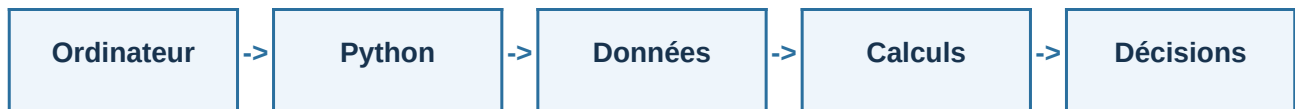
Créer des graphiques avec Matplotlib et communiquer avec des instruments de mesure par liaison série ou réseau, notamment avec des commandes SCPI ou une carte Arduino.

Séance 1

Premiers programmes Python

De la machine aux données, calculs et décisions

Nous commencerons dans la console Python, utilisée comme une calculatrice, puis nous enregistrerons les mêmes instructions dans un véritable script :



Le fil rouge sera une mesure de température : la saisir, la convertir, effectuer un calcul et choisir un message selon sa valeur.

À la fin de la séance, vous devez être capables de lire et d'écrire ce type de script simple.

Cette séance pose les bases utilisées dans tout le cours : exécuter un programme, représenter une donnée, effectuer un calcul, communiquer avec l'utilisateur et prendre une décision. Les séances suivantes permettront d'organiser ces traitements et de les appliquer à des séries de mesures.

Architecture matérielle

Un programme utilise plusieurs ressources de l'ordinateur :

	Élément	Rôle principal
CPU		exécuter les instructions
RAM		accueillir le programme et les données en cours d'utilisation
Stockage		conserver durablement programmes et fichiers
Périphériques		échanger avec l'extérieur



La RAM est une mémoire de travail rapide mais volatile : son contenu disparaît lorsque la machine est éteinte. Le CPU exécute des instructions machine ; dans notre cas, l'interpréteur fait le lien entre le code Python et leur exécution. Les périphériques regroupent notamment le clavier, l'écran et les capteurs.

Architecture logicielle

Le **système d'exploitation** gère la mémoire, les fichiers, les périphériques et le lancement des applications.

Dans son usage courant, un programme Python suit cette chaîne :



```
print("Première mesure")
```

On oppose souvent les programmes compilés, traduits avant leur exécution, aux programmes interprétés, pris en charge par un interpréteur. Cette opposition est un modèle simplifié : l'implémentation de référence de Python produit également une représentation intermédiaire appelée bytecode.

Console interactive et script

Python peut être utilisé de deux manières complémentaires :

Console interactive

tester rapidement une instruction
obtenir un résultat immédiat
expérimenter

Script `.py`

conserver un programme
enchaîner plusieurs traitements
modifier et réutiliser le code

```
>>> temperature = 20.5 >>> temperature + 273.15 293.65
```

Dans Thonny, le **Shell** sert à essayer une instruction ; l'éditeur sert à construire puis conserver `mesure.py`.

Le même script peut aussi être lancé avec `python mesure.py` dans un terminal (`py mesure.py` sur certaines installations Windows).

Trouver une information fiable

Personne ne mémorise toute la bibliothèque Python. Deux réflexes sont utiles :

```
help(print) help(float)
```

- `help()` décrit une fonction, un type ou une valeur disponible dans l'environnement ;
- docs.python.org est la documentation officielle du langage et de sa bibliothèque standard.

Pour une fonction externe, par exemple NumPy ou pyserial, il faut consulter la documentation de la bibliothèque concernée.

Variables et affectation

Une variable permet de donner un nom à une valeur pour la réutiliser :

```
temperature = 20.5
```

```
temperature = 20.5 ■■■■■■■■■■■■ ■■■■ nom valeur
```

= réalise une **affectation**. Une nouvelle affectation remplace la valeur associée au nom.

Une affectation n'est pas une égalité mathématique. Python évalue l'expression située à droite, puis associe son résultat au nom situé à gauche. Des noms comme `temperature` ou `nombre_mesures` rendent le programme plus compréhensible que des noms génériques comme `x` ou `a`.

Types de données

Le type décrit la nature d'une valeur et détermine les opérations possibles :

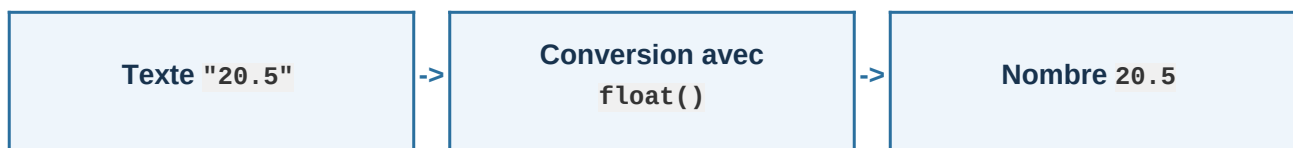
Type	Nature	Exemple
<code>int</code>	nombre entier	<code>299_792_458</code>
<code>float</code>	nombre à virgule flottante	<code>1.602e-19</code>
<code>str</code>	chaîne de caractères	<code>"COM3"</code>
<code>bool</code>	résultat logique	<code>True</code> ou <code>False</code>

```
print(type(20.5)) # <class 'float'> print(type(20 > 10)) # <class 'bool'>
```

Python utilise un typage dynamique : le type est associé à la valeur et déterminé pendant l'exécution. La fonction `type()` permet d'observer le type d'une donnée, ce qui est notamment utile pour comprendre une erreur de conversion.

Conversions explicites

Une donnée doit parfois changer de représentation avant d'être utilisée :



```
temperature = float("20.5") nombre_mesures = int("12") message = str(20.5)
```

`float()`, `int()` et `str()` produisent une nouvelle valeur du type demandé. Une conversion échoue si le contenu n'est pas compatible : `float("vingt")` provoque une erreur.

Dans un programme Python, le séparateur décimal est toujours le point : `20.5`, et non `20,5`.

Opérateurs arithmétiques

Python fournit les opérations usuelles :

Expression	Opération	Résultat pour <code>a = 7, b = 2</code>
<code>a + b</code>	addition	9
<code>a - b</code>	soustraction	5
<code>a * b</code>	multiplication	14
<code>a / b</code>	division	3.5
<code>a // b</code>	quotient arrondi vers le bas	3
<code>a % b</code>	reste	1
<code>a ** b</code>	puissance	49

Les parenthèses permettent de rendre l'ordre des calculs explicite.

L'opérateur `//` est parfois appelé « division entière », mais cette expression peut être trompeuse : le résultat est arrondi vers le bas. Ainsi, `-7 // 2` produit `-4`. Le reste `%` permet notamment de tester si un entier est pair avec la condition `n % 2 == 0`.

Affectation, comparaison et mise à jour

Il faut distinguer la modification d'une variable et la comparaison de deux valeurs :

Écriture	Rôle	Résultat
<code>temperature = 20</code>	affecter	la variable reçoit 20
<code>temperature == 20</code>	comparer	True ou False
<code>temperature != 20</code>	comparer	True ou False

Deux écritures possibles pour la même mise à jour :

```
nombre = nombre + 1 # ou, sous forme abrégée : nombre += 1
```

Autres comparaisons : `<`, `<=`, `>` et `>=`.

Précision des nombres flottants

Les nombres décimaux ne peuvent pas tous être représentés exactement en mémoire :

```
resultat = 0.1 + 0.2 print(resultat)
```

```
0.30000000000000004
```

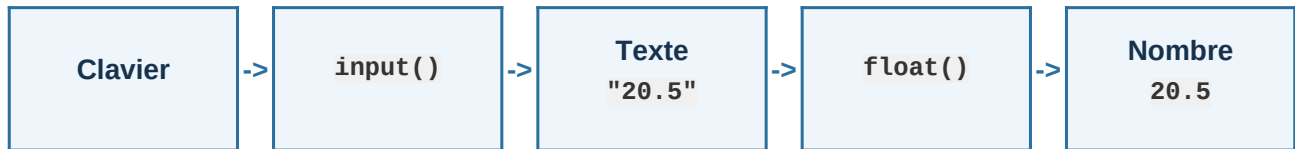
Un `float` représente donc une valeur numérique avec une précision limitée. Cette approximation est normale et doit être prise en compte lors des comparaisons.

De nombreux nombres décimaux ne possèdent pas de représentation binaire finie. Pour comparer deux résultats calculés, on utilise parfois une tolérance, par exemple `abs(a - b) < 1e-9`, plutôt qu'une égalité stricte avec `==`.

Saisie et conversion

`input()` permet de recevoir une information saisie au clavier. Son résultat est toujours une chaîne de caractères :

```
saisie = input("Température : ") temperature = float(saisie)
```



Après la conversion, `temperature` peut être utilisée dans un calcul.

`input()` interrompt le programme jusqu'à la validation de la saisie. Une saisie invalide comme vingt provoque une erreur pendant la conversion. La validation des données sera approfondie plus tard ; pour le moment, on suppose que l'utilisateur fournit une valeur compatible.

Affichage et f-strings

`print()` affiche une information. Une f-string permet d'intégrer une valeur dans un texte :

```
temperature = 20.456 print(f"Température : {temperature:.1f} °C")
```

```
Température : 20.5 °C
```

Le format `.1f` affiche la valeur arrondie à un chiffre après la virgule. Il modifie l'affichage, pas la valeur utilisée dans les calculs.

Prendre une décision

Un programme peut adapter son comportement à la valeur d'une mesure :

```
if temperature < 0: print("Risque de gel") elif temperature < 30:  
print("Température dans la plage prévue") else: print("Seuil supérieur dépassé")
```

Les conditions sont évaluées dans l'ordre. Python exécute la première branche dont la condition est vraie ; `else` traite les autres cas. L'indentation délimite les instructions de chaque branche.

`if` introduit le premier test, `elif` ajoute un test lorsque les précédents sont faux et `else` traite tous les autres cas. Une seule branche de cette structure est exécutée. Si le second test est atteint, Python sait déjà que la température est supérieure ou égale à zéro.

Combiner des conditions

Dans une condition, les opérateurs logiques permettent de combiner plusieurs tests :

	Opérateur	Signification
	<code>and</code>	les deux conditions sont vraies
	<code>or</code>	au moins une condition est vraie
	<code>not</code>	inverse une valeur logique

```
if 0 <= temperature < 30: print("Température dans la plage prévue")
```

Python autorise cette comparaison enchaînée. Pour des tests de nature différente, on utilise `and`, `or` et `not`. Si une condition devient difficile à lire, il est préférable de la décomposer.

Comprendre les erreurs

Une erreur fait partie du travail de programmation. Il faut d'abord identifier sa catégorie :

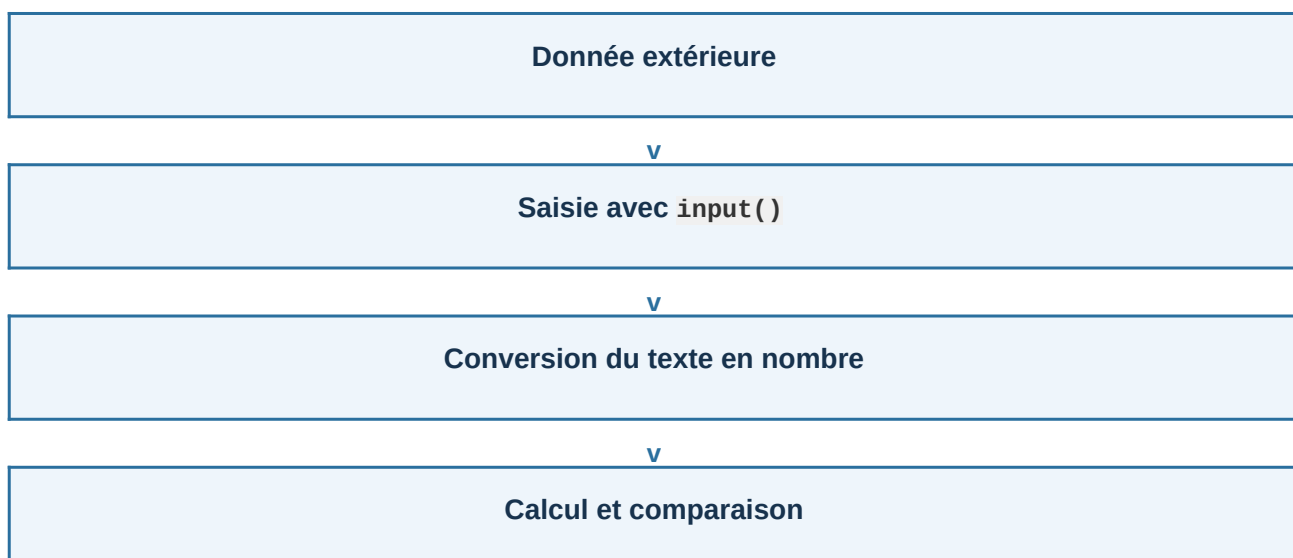
	Catégorie	Exemple
syntaxe		parenthèse ou deux-points absents
exécution		conversion impossible avec <code>float("abc")</code>
logique		seuil incorrect dans une condition

Les erreurs de syntaxe et d'exécution produisent généralement un message indiquant la ligne et la nature du problème. Une erreur de logique doit être repérée en vérifiant les résultats.

Une erreur de syntaxe empêche Python de comprendre le programme. Une erreur d'exécution apparaît pendant une opération impossible. Une erreur de logique est plus discrète : le programme s'exécute, mais produit un mauvais résultat. Dans un traceback, la dernière ligne donne généralement le type d'erreur.

Synthèse de la séance

Nous pouvons maintenant construire une première chaîne de traitement :



v

Décision avec `if`

v

Résultat affiché

Le programme sait recevoir une donnée, la convertir, la mémoriser, effectuer un calcul, prendre une décision et afficher un résultat. La séance suivante permettra d'organiser les traitements avec des fonctions et de manipuler plusieurs valeurs.

Exercices — Séance 1 : Premiers programmes Python

Ex. 1 — Premier script Créer un fichier `bonjour.py` qui affiche votre nom et votre filière, puis l'exécuter depuis un terminal.

Ex. 2 — Calculatrice d'opérations Demander deux nombres à l'utilisateur (`input`) et afficher le résultat de `+` `-` `*` `/` `//` `%` `**` entre eux, avec des f-strings lisibles. Pour ce premier exercice, choisir un second nombre non nul.

Ex. 3 — Conversions de types Demander une chaîne comme `"3.14"`, la convertir en `float`, puis afficher la valeur avec deux chiffres après la virgule à l'aide d'une f-string.

Ex. 4 — Précision flottante Calculer `0.1 + 0.2`, comparer à `0.3` avec `==` puis avec une tolérance (`abs(a-b) < 1e-9`). Expliquer la différence en commentaire.

Ex. 5 — Classification d'une mesure Demander une pression en bar et afficher : "normale" si `1 <= pression <= 2`, "alerte" si `0 <= pression < 1` ou `2 < pression <= 3`, et "danger" si `pression > 3`. Une valeur négative doit être signalée comme invalide.

Ex. 6 — Autonomie d'une batterie Demander la capacité d'une batterie en ampères-heures et le courant consommé en ampères. Calculer l'autonomie théorique `capacite / courant`, l'afficher avec une décimale, puis indiquer "faible" sous 2 h, "moyenne" de 2 h à moins de 5 h, ou "élevée" à partir de 5 h. Signaler le cas d'un courant nul ou négatif.

Séance 2

Organiser et répéter les traitements

Fonctions, objets, séquences, textes et boucles

Un programme devient rapidement difficile à maintenir lorsqu'il répète du code ou manipule de nombreuses valeurs séparément.

À la fin de la séance, vous devez être capables de découper un programme en fonctions, d'utiliser des objets existants, de regrouper des données et de répéter automatiquement un traitement.

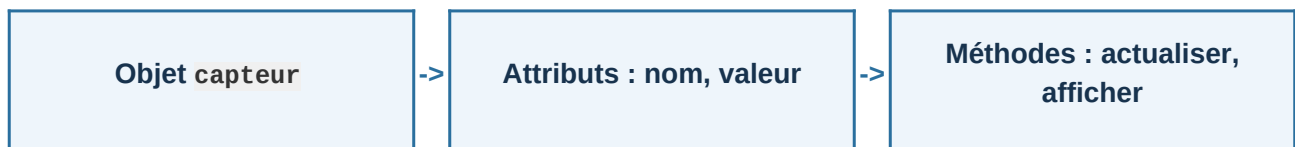
Cette séance fait passer du petit programme linéaire à un programme structuré. Les fonctions organisent les traitements, les séquences regroupent les données et les boucles permettent d'appliquer une même opération à plusieurs valeurs.

La POO rassemble données et comportements

La **programmation orientée objet** (POO) organise un programme autour d'objets qui regroupent :

- des **attributs** : les données propres à l'objet ;
- des **méthodes** : les opérations que l'objet sait effectuer.

Un objet représente ainsi une entité du programme avec un état et des comportements.



La POO est particulièrement utile lorsqu'un programme manipule plusieurs éléments de même nature : capteurs, instruments, fichiers ou graphiques.

La POO associe les données et les traitements qui leur correspondent au lieu de les disperser dans le programme. Une classe définit cette organisation commune ; les objets créés à partir d'elle portent leurs propres valeurs.

Une classe sert de modèle aux objets

Une **classe** décrit la structure commune de ses futurs objets. Pour les exemples suivants, on suppose qu'une bibliothèque fournit cette classe :

```
CapteurTemperature(nom, valeur_initiale)
```

Élément défini par la classe

Exemple

constructeur	<code>CapteurTemperature(...)</code>
attributs	<code>.nom</code> , <code>.valeur</code>
méthodes	<code>.actualiser()</code> , <code>.afficher()</code>

La classe constitue le modèle commun ; les valeurs particulières appartiendront à chaque objet créé.

La documentation d'une classe présente généralement son constructeur, ses attributs et ses méthodes. Elle indique les arguments attendus, les types de valeurs et les effets produits.

Instancier crée un nouvel objet

Créer un objet à partir d'une classe s'appelle une **instanciation** :

```
capteur_salle = CapteurTemperature("Salle A", 20.5)
```



`capteur_salle` est une **instance** de `CapteurTemperature`. Dans ce cours, il faudra surtout savoir instancier et utiliser des classes existantes, pas écrire leur définition.

Le constructeur s'appelle comme une fonction. Ses arguments fournissent les valeurs nécessaires à l'initialisation. L'objet obtenu possède alors les attributs et les méthodes prévus par la classe.

Chaque instance conserve son propre état

Deux appels du constructeur créent deux objets indépendants :

```
capteur_salle = CapteurTemperature("Salle A", 20.5) capteur_dehors =
CapteurTemperature("Extérieur", 12.8) capteur_salle.valeur = 21.0
print(capteur_salle.valeur) # 21.0 print(capteur_dehors.valeur) # 12.8
```

Modifier `capteur_salle` ne modifie pas `capteur_dehors`.

On accède aux attributs et aux méthodes d'un objet avec le **point** :

```
objet.attribut objet.methode()
```

Chaque instance possède son identité et son propre ensemble d'attributs. Les objets peuvent donc évoluer séparément, même lorsqu'ils ont été créés à partir de la même classe.

Les attributs décrivent l'état d'un objet

Les attributs sont des variables associées à une instance. Ils sont souvent initialisés par le constructeur :

```
capteur = CapteurTemperature("Salle A", 20.5) print(capteur.nom) # Salle A
print(capteur.valeur) # 20.5
```

Un attribut modifiable peut ensuite recevoir une nouvelle valeur :

```
capteur.valeur = 21.3 print(capteur.valeur) # 21.3
```

La documentation précise le nom, le rôle, le type et l'unité éventuelle de chaque attribut. Il ne faut pas inventer un attribut qui n'est pas prévu par la classe.

Tous les attributs ne sont pas nécessairement destinés à être modifiés directement. Certaines classes imposent de passer par une méthode afin de contrôler la nouvelle valeur ou de maintenir la cohérence de l'objet.

Les méthodes font agir l'objet

Une méthode est une fonction attachée à un objet. Elle peut consulter ou modifier ses attributs :

```
capteur = CapteurTemperature("Salle A", 20.5) capteur.actualiser(21.3)
print(capteur.valeur) # 21.3
```

Une méthode peut aussi produire un affichage ou renvoyer un résultat :

```
capteur.afficher() # Salle A : 21.3 °C
```

Dans `capteur.actualiser(21.3)`, l'objet placé avant le point est celui sur lequel la méthode agit.

La méthode reçoit implicitement l'objet concerné. Son effet et sa valeur de retour sont décrits dans la documentation. Certaines méthodes modifient l'objet, tandis que d'autres renvoient une nouvelle valeur sans changer l'état initial.

En Python, toutes les valeurs sont des objets

Les objets ne concernent pas seulement les classes fournies par une bibliothèque. Les nombres, textes et listes sont eux aussi des objets :

```
temperature = 20.5 message = "mesure terminée" mesures = [18.2, 19.1]
print(type(temperature)) # <class 'float'> print(type(message)) # <class 'str'>
print(type(mesures)) # <class 'list'>
```

Leur type détermine les opérations et méthodes disponibles :

```
message.upper() # "MESURE TERMINÉE" mesures.append(20.0) # modifie la liste
```

Nous utiliserons cette même notation avec NumPy, Matplotlib et pyserial.

Inspecter un objet

Python permet d'interroger un objet lorsque l'on ne connaît pas exactement son contenu :

```
type(capteur) isinstance(capteur, CapteurTemperature) dir(capteur)
help(capteur.actualiser)
```

Outil	Information obtenue
<code>type(objet)</code>	classe de l'objet
<code>isinstance(objet, Classe)</code>	vérification du type attendu
<code>dir(objet)</code>	noms des attributs et méthodes
<code>help(...)</code>	documentation disponible

`dir()` peut produire une longue liste contenant des noms techniques. Il sert surtout à repérer un nom, puis `help()` ou la documentation explique comment l'utiliser.

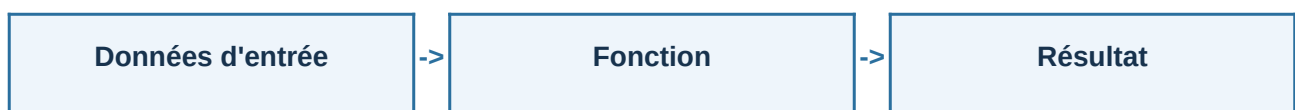
Dans un environnement comme Thonny, l'autocomplétion après le point complète cette inspection. Ces outils aident à explorer, mais la documentation officielle reste la référence pour connaître les paramètres, les effets et les valeurs renvoyées.

Organiser le code avec des fonctions

Copier-coller un même calcul à plusieurs endroits augmente le risque d'erreur et rend les modifications difficiles.

Une fonction regroupe un traitement sous un nom :

```
def convertir_c_en_k(temperature_c): return temperature_c + 273.15
temperature_k = convertir_c_en_k(20.0)
```



La définition avec `def` enregistre le traitement mais ne l'exécute pas encore. L'appel de la fonction déclenche son exécution. Un nom de fonction commence généralement par un verbe qui décrit l'action réalisée.

Paramètres et arguments

Un **paramètre** est un nom utilisé dans la définition. Un **argument** est la valeur transmise lors de l'appel.

```
def convertir_c_en_k(temperature_c): # paramètre return temperature_c + 273.15
resultat = convertir_c_en_k(20.0) # argument
```

Le paramètre permet d'utiliser le même traitement avec des valeurs différentes.

Valeurs par défaut

Un paramètre peut recevoir une valeur utilisée lorsque l'appel ne fournit pas l'argument correspondant :

```
def afficher_mesure(valeur, unite="°C"): print(f"Mesure : {valeur} {unite}")
afficher_mesure(20.5) afficher_mesure(293.65, "K")
```

Les paramètres obligatoires sont placés avant ceux qui possèdent une valeur par défaut.

Une valeur par défaut convient à un cas fréquent, mais l'appelant peut la remplacer. Les paramètres rendent une fonction générale sans l'obliger à lire directement des variables globales ou à demander elle-même les données à l'utilisateur.

Retourner ou afficher

`return` fournit un résultat au reste du programme ; `print()` produit seulement un affichage.

```
def calculer_energie(puissance, duree): return puissance * duree
energie = calculer_energie(12.0, 3.0) print(f"Énergie : {energie:.1f} Wh")
```

Le résultat retourné peut être mémorisé, comparé ou utilisé dans un autre calcul.

Une fonction sans `return` explicite renvoie la valeur spéciale `None`. `return` interrompt également l'exécution de la fonction. Séparer le calcul de l'affichage rend généralement la fonction plus facile à réutiliser et à tester.

Portée des variables

Une variable créée dans une fonction est **locale** : elle n'est accessible que pendant l'exécution de cette fonction.

```
def convertir_c_en_k(temperature_c): resultat = temperature_c + 273.15
return resultat
temperature = 20.0
```

`resultat` est locale à la fonction ; `temperature` est définie dans le programme principal.

Une fonction peut lire certaines variables définies à l'extérieur, mais cette dépendance est souvent difficile à repérer. Il est préférable de transmettre les données par les paramètres et de récupérer les résultats avec `return`. L'emploi de variables globales doit rester limité et justifié.

Regrouper des valeurs avec une liste

Une série de mesures ne doit pas être stockée dans une succession de variables indépendantes.

```
mesures = [18.2, 19.1, 20.0, 19.7]
```

Une liste est une séquence **ordonnée** et **modifiable** :

```
mesures[0] = 18.4 mesures.append(20.2)
```

Elle peut contenir un nombre variable d'éléments.

Les éléments d'une liste sont placés entre crochets et séparés par des virgules. Python autorise le mélange de types, mais une série scientifique reste généralement homogène. `append()` ajoute un élément à la fin de la liste.

Tuples et dépaquetage

Un tuple est une séquence ordonnée qui ne peut pas être modifiée après sa création.

```
position = (48.85, 2.35) latitude, longitude = position
```

Liste	Tuple
<code>[18.2, 19.1]</code>	<code>(48.85, 2.35)</code>
modifiable	non modifiable
série évolutive	ensemble de valeurs associé

L'impossibilité de modifier un tuple est appelée immutabilité. Le dépaquetage affecte chaque élément à une variable ; le nombre de variables doit correspondre au nombre d'éléments. Les tuples sont courants pour représenter des coordonnées ou retourner plusieurs résultats.

Indexer une séquence

Chaque élément possède une position appelée **index**. Le premier index vaut zéro.

```
valeur 10 20 30 40 index 0 1 2 3 négatif -4 -3 -2 -1
```

```
valeurs = [10, 20, 30, 40] premiere = valeurs[0] derniere = valeurs[-1]
```

Un index inexistant provoque une `IndexError`.

Extraire une partie de séquence

Le **slicing** sélectionne une portion sans modifier la séquence d'origine :

```
valeurs = [10, 20, 30, 40, 50] valeurs[1:4] # [20, 30, 40] valeurs[:2] # [10, 20]  
valeurs[::2] # [10, 30, 50]
```

Dans `[debut:fin]`, la borne de début est incluse et celle de fin est exclue.

Le troisième nombre éventuel indique le pas. Une borne omise signifie le début ou la fin de la séquence. Les mêmes principes s'appliquent aux listes, aux tuples, aux chaînes de caractères et, plus tard, aux tableaux NumPy.

Fonctions utiles sur les séquences

Python fournit des opérations courantes qui évitent de réécrire une boucle :

```
mesures = [18.2, 19.1, 20.0] len(mesures) # nombre d'éléments sum(mesures) # somme
min(mesures) # minimum max(mesures) # maximum sorted(mesures) # nouvelle liste
triée
```

Le test `19.1 in mesures` indique si une valeur appartient à la séquence.

`sorted()` produit une nouvelle liste et ne modifie pas la séquence fournie. À l'inverse, la méthode `list.sort()` modifie directement une liste. `sum()`, `min()` et `max()` supposent que les éléments concernés sont compatibles entre eux.

Chaînes de caractères

Une chaîne `str` est une séquence de caractères. Elle peut être indexée et découpée :

```
capteur = "TEMP-01" capteur[0] # "T" capteur[:4] # "TEMP"
```

Quelques caractères possèdent une écriture spéciale : `\n` représente un retour à la ligne et `\t` une tabulation.

Pour un chemin Windows contenant des antislashes, une chaîne brute évite de les doubler :

```
chemin = r"C:\mesures\essai1.csv"
```

Les chaînes sont non modifiables : une transformation produit une nouvelle chaîne.

Nettoyer et découper du texte

Les méthodes de chaînes préparent notamment les données lues dans un fichier :

```
ligne = " temperature;20,5 " ligne = ligne.strip() ligne = ligne.replace(",", ".")
morceaux = ligne.split(";") nom = morceaux[0] valeur = float(morceaux[1])
```

`join()` effectue l'opération inverse en réunissant plusieurs chaînes avec un séparateur.

`strip()` retire les espaces et fins de ligne aux extrémités. `replace()` remplace toutes les occurrences recherchées. `split()` découpe et renvoie une liste. La méthode `find()` recherche une sous-chaîne et renvoie son index, ou `-1` si elle est absente.

Répéter un traitement avec `for`

Une boucle `for` parcourt successivement les éléments d'une séquence :

```
mesures = [18.2, 19.1, 20.0] for mesure in mesures: print(f"{mesure:.1f} °C")
```

À chaque tour, `mesure` reçoit l'élément suivant. Le bloc indenté est exécuté une fois par élément.

Il n'est pas nécessaire de gérer manuellement l'index lorsque seule la valeur est utile. Le nom de la variable de boucle doit décrire l'élément parcouru. Une chaîne, un tuple et de nombreuses autres structures peuvent également être parcourus.

Compter et accumuler

`range()` produit une suite d'entiers, notamment pour répéter une opération un nombre connu de fois :

```
for numero in range(5): print(numero) # de 0 à 4
```

Une variable peut accumuler progressivement un résultat :

```
total = 0 for mesure in mesures: total += mesure
```

`range(debut, fin, pas)` exclut la borne de fin, comme le slicing.

Répéter avec `while`

Une boucle `while` répète un bloc tant que sa condition reste vraie :

```
tentatives = 0 while tentatives < 3: print(f"Tentative {tentatives + 1}")
tentatives += 1
```

Elle convient lorsque le nombre de répétitions dépend d'une condition. La condition doit pouvoir devenir fausse pour éviter une boucle infinie.

Modifier le parcours d'une boucle

Deux instructions permettent de modifier ponctuellement le déroulement :

- `break` arrête immédiatement la boucle ;
- `continue` passe directement au tour suivant.

```
for mesure in mesures: if mesure < 0: continue if mesure > 30: break print(mesure)
```

Elles restent utiles lorsque leur rôle est simple et clairement identifiable.

Synthèse de la séance

Nous disposons maintenant de trois outils complémentaires :

```
organiser un traitement → fonction regrouper des données → liste, tuple, str  
répéter une opération → boucle for ou while
```



Les paramètres font entrer les données dans une fonction et `return` en fait sortir le résultat. Les séquences regroupent plusieurs valeurs et les boucles les parcourent. Ces outils permettront, à la séance suivante, de lire et d'analyser des données conservées dans des fichiers.

Exercices — Séance 2 : Organiser et répéter les traitements

Ex. 1 — Fonction sans `max()` Écrire une fonction `maximum(liste)` qui renvoie le plus grand élément d'une liste non vide, sans utiliser `max()`, à l'aide d'une boucle.

Ex. 2 — Fonctions de conversion Écrire une fonction `convertir_depuis_m(valeur, unite="m")`. La valeur d'entrée est exprimée en mètres et l'unité de sortie peut être `"m"`, `"cm"` ou `"km"`. La fonction doit retourner le résultat et signaler une unité inconnue.

Ex. 3 — Tuple et dépaquetage Créer un tuple de coordonnées GPS (`latitude, longitude`), le dépaqueter dans deux variables, et afficher un message formaté.

Ex. 4 — Découpage de texte À partir d'une phrase saisie par l'utilisateur, afficher : les 5 premiers caractères, les 5 derniers, un mot sur deux (via `split()` et slicing de liste).

Ex. 5 — Nettoyage de texte À partir d'une chaîne du type `" Alice ; Bob ;Charlie "`, obtenir la liste `["Alice", "Bob", "Charlie"]` sans espaces superflus, en utilisant `strip()`, `split()` et une boucle.

Ex. 6 — Recherche avec `while/break` Écrire un programme qui parcourt une liste de mesures avec `while` et s'arrête (`break`) dès qu'une valeur dépasse un seuil, en affichant sa position (utiliser `continue` pour ignorer les valeurs négatives). Veiller à faire avancer l'indice à chaque tour, y compris avant `continue`.

Séance 3

Exploiter des données scientifiques

Fichiers, CSV et calcul avec NumPy

À la fin d'une expérience, fermer Python ne doit pas faire disparaître les mesures. Il faut les enregistrer dans un format que l'on saura relire quelques jours plus tard, éventuellement avec un autre logiciel.

À la fin de la séance, vous devez être capables de lire un fichier de mesures, de convertir son contenu et d'effectuer des calculs simples avec NumPy.

Cette séance construit la chaîne qui relie un fichier à une analyse numérique. Elle réutilise les conversions, les listes et les boucles, puis introduit les tableaux NumPy adaptés au calcul scientifique.

Modules et bibliothèques

Une bibliothèque fournit du code déjà développé pour un domaine particulier. C'est le fonctionnement normal de Python : on ne réécrit pas soi-même un lecteur CSV ou une fonction de calcul numérique.

	Élément	Rôle
module		unité de code importable, souvent un fichier <code>.py</code>
package		ensemble organisé de modules
bibliothèque standard		installée avec Python
bibliothèque externe		installée séparément

Exemples : `math` et `csv` sont fournis avec Python ; `numpy`, `matplotlib` et `serial` sont installés séparément.

Dans l'usage courant, les mots package et bibliothèque sont parfois employés de manière souple. L'idée essentielle est la réutilisation : on s'appuie sur des fonctions testées au lieu de reconstruire tous les outils nécessaires. La documentation officielle et `help()` permettent ensuite de vérifier les arguments attendus et les valeurs renvoyées.

Installer et importer

Une bibliothèque externe est installée une fois dans l'environnement Python :

```
python -m pip install numpy
```

Elle est ensuite importée dans chaque programme qui l'utilise :

```
import math from math import sqrt import numpy as np
```

Avec `import numpy as np`, `np` est un alias court et conventionnel.

Un paquet externe doit être choisi avec soin sur PyPI : le nom saisi dans `pip install` doit être vérifié avant l'installation.

```
import math
```

 conserve le nom du module dans les appels, par exemple `math.sqrt(16)`. `from math import sqrt` permet d'écrire directement `sqrt(16)`, mais rend l'origine de la fonction moins visible. L'installation doit être faite dans le même environnement que celui qui exécute le programme.

Conserver les données

Une variable existe seulement pendant l'exécution du programme. Un fichier permet de retrouver les données après son arrêt.

```
programme en cours → variables en RAM
```

 expérience terminée → données conservées dans un fichier

Les fichiers permettent également d'échanger des mesures avec un tableur, un instrument ou un autre programme.

Formats et chemins

Un fichier **texte** contient des caractères directement lisibles, tandis qu'un fichier **binaire** suit une organisation qui dépend de son format.

Chemin relatif	Chemin absolu
<code>data/mesures.csv</code>	<code>C:/projet/data/mesures.csv</code>
dépend du dossier de travail	désigne un emplacement complet

Le format du fichier et son extension ne suffisent pas : le programme doit connaître l'organisation réelle des données.

Pour des mesures tabulées, la tabulation `\t` ou le point-virgule `;` sont souvent plus pratiques qu'une virgule lorsque les données circulent aussi dans un tableur configuré en français.

Les fichiers `.txt`, `.csv` et `.json` sont des formats textuels. Les images, certains fichiers de tableur et de nombreux formats scientifiques sont binaires. Un chemin relatif est interprété à partir du dossier de travail courant, qui peut différer du dossier où se trouve le script.

Ouvrir un fichier avec `with`

```
with open("mesures.txt", "r", encoding="utf-8") as fichier: contenu = fichier.read()
```

Le bloc `with` ferme automatiquement le fichier à sa sortie, même si une erreur survient.

Mode	Action
r	lire un fichier existant
w	écrire en remplaçant le contenu
a	ajouter à la fin
x	créer un nouveau fichier

Le mode de lecture `r` est utilisé par défaut. Le paramètre `encoding="utf-8"` indique comment les octets du fichier texte sont convertis en caractères. Le mode `w` doit être utilisé avec prudence car il efface le contenu précédent.

Lire un fichier ligne par ligne

Une boucle permet de traiter un fichier sans charger tout son contenu en une seule fois :

```
with open("mesures.txt", "r", encoding="utf-8") as fichier:
    for ligne in fichier:
        texte = ligne.strip()
        mesure = float(texte)
        print(mesure)
```



Chaque ligne contient généralement un caractère de fin de ligne retiré par `strip()`. La conversion peut échouer si une ligne est vide ou mal formée. Dans une application réelle, il faut définir une stratégie pour ces données invalides.

Structure d'un fichier CSV

Un CSV représente des données tabulaires dans un fichier texte :

```
temps;temperature 0;20.1 1;20.4 2;20.9
```

Ligne	Rôle
première ligne	noms des colonnes
lignes suivantes	observations
;	séparateur utilisé ici

Les valeurs visibles comme des nombres sont d'abord lues comme du texte.

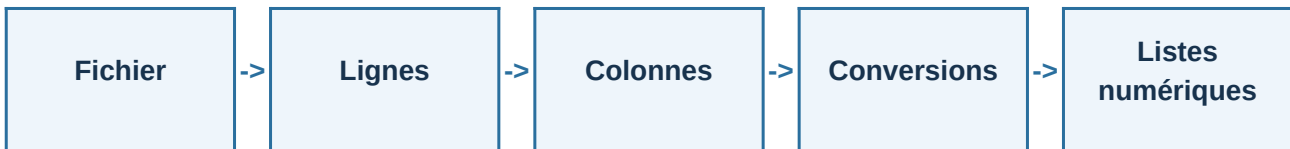
CSV signifie *Comma-Separated Values*, mais le séparateur dépend des usages et de la configuration régionale. En France, le point-virgule est fréquent afin de ne pas le confondre avec la virgule décimale. Le séparateur doit être indiqué au lecteur lorsque ce n'est pas une virgule.

Lire et convertir un CSV

```
import csv
temps = []
temperatures = []
with open("mesures.csv", newline="", encoding="utf-8") as fichier:
    lecteur = csv.reader(fichier, delimiter=";")
```

```
next(lecteur) # ignorer l'en-tête
for ligne in lecteur:
    temps.append(float(ligne[0]))
    temperatures.append(float(ligne[1]))
```

`next(lecteur)` récupère la ligne suivante et avance le lecteur. Ici, le premier appel consomme donc la ligne d'en-tête avant le début de la boucle.



`csv.reader` gère correctement les séparateurs et les champs éventuellement protégés par des guillemets. `next(lecteur)` consomme la première ligne. Cette version suppose que le fichier possède un en-tête et que chaque ligne contient deux nombres valides.

Écrire un résultat

Le mode `w` permet de produire un rapport ou un nouveau fichier de données :

```
moyenne = 20.47
with open("rapport.txt", "w", encoding="utf-8") as fichier:
    fichier.write(f"Moyenne : {moyenne:.2f} °C\n")
```

`write()` attend une chaîne de caractères et n'ajoute pas automatiquement de retour à la ligne.

L'écriture permet de conserver un résultat ou de le transmettre à un autre outil. Pour produire un véritable CSV, il est préférable d'utiliser `csv.writer` plutôt que d'assembler manuellement les séparateurs.

NumPy et le calcul scientifique

Une liste Python peut stocker des mesures, mais elle n'est pas spécialisée dans le calcul numérique.

```
import numpy as np
temperatures = np.array([20.1, 20.4, 20.9])
```

Un `ndarray` NumPy fournit :

- des données numériques homogènes ;
- des tableaux à plusieurs dimensions ;
- des opérations appliquées à l'ensemble des valeurs ;
- des fonctions de calcul scientifique.

NumPy stocke les éléments de manière régulière et exécute de nombreuses opérations dans du code optimisé. Pour de grandes séries numériques, cela rend les calculs plus rapides et leur écriture souvent plus concise qu'avec des boucles Python.

Créer un tableau NumPy

```
import numpy as np
a = np.array([1, 2, 3, 4])
zeros = np.zeros(4)
indices = np.arange(0, 10, 2)
temps = np.linspace(0, 1, 5)
```

Fonction	Usage
<code>np.array</code>	convertir une séquence existante
<code>np.zeros</code>	initialiser avec des zéros
<code>np.arange</code>	créer une progression avec un pas
<code>np.linspace</code>	répartir un nombre donné de valeurs

Dimensions et type des éléments

```
tableau = np.array([[1, 2, 3], [4, 5, 6]])
```

Propriété	Valeur	Signification
<code>tableau.shape</code>	<code>(2, 3)</code>	2 lignes et 3 colonnes
<code>tableau.ndim</code>	2	2 dimensions
<code>tableau.size</code>	6	6 éléments au total
<code>tableau.dtype</code>	type entier	type commun des éléments

Un `ndarray` possède une forme et un type d'éléments homogène.

Accéder aux valeurs

Pour un tableau à deux dimensions, les indices sont donnés dans l'ordre **ligne, colonne** :

```
tableau[0, 1] # élément de la ligne 0, colonne 1
tableau[1, :] # deuxième ligne
tableau[:, 0] # première colonne
tableau[0:2, 1:3]
```

Le symbole `:` signifie ici « tous les indices » de la dimension concernée.

L'indexage commence à zéro comme pour les listes. NumPy étend le slicing à plusieurs dimensions en séparant les sélections par une virgule.

Charger directement un fichier tabulé

Maintenant que l'indexation d'un tableau à deux dimensions est connue, NumPy peut charger puis séparer les colonnes d'un fichier numérique régulier :

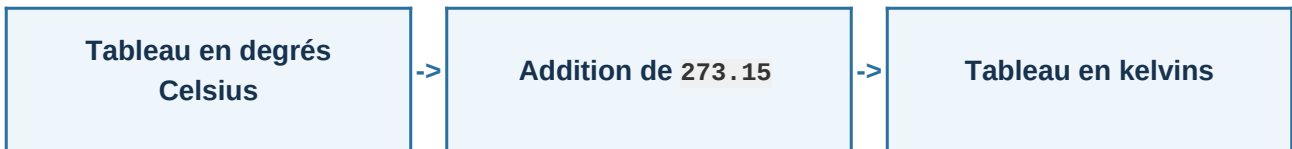
```
import numpy as np
donnees = np.loadtxt("mesures.csv", delimiter=";", skiprows=1,
)
temps = donnees[:, 0]
temperatures = donnees[:, 1]
```

`skiprows=1` ignore la ligne d'en-tête. `donnees[:, 0]` sélectionne toute la première colonne et `donnees[:, 1]` toute la deuxième. Cette méthode suppose que les autres lignes contiennent les colonnes numériques attendues.

Opérations vectorisées

NumPy applique une opération à tous les éléments d'un tableau :

```
temperatures_c = np.array([18.2, 19.1, 20.0]) temperatures_k = temperatures_c + 273.15
```



La vectorisation remplace ici une boucle explicite par une expression portant sur le tableau entier.

Statistiques descriptives

```
mesures = np.array([18.2, 19.1, 20.0, 19.7]) mesures.mean() # moyenne  
mesures.std() # écart-type mesures.min() # minimum mesures.max() # maximum  
mesures.sum() # somme
```

La moyenne résume le niveau central ; l'écart-type décrit la dispersion autour de cette moyenne.

`std()` calcule ici l'écart-type de la population décrite par le tableau. Dans un cours de statistique, une correction peut être appliquée lorsqu'un échantillon est utilisé pour estimer une population. Cette distinction n'est pas nécessaire pour les premiers traitements descriptifs de cette séance.

Calculer suivant un axe

```
tableau = np.array([[1, 2, 3], [4, 5, 6]]) tableau.mean(axis=0) # [2.5, 3.5, 4.5]  
tableau.mean(axis=1) # [2.0, 5.0]
```

	Calcul	Résultat
<code>axis=0</code>		une moyenne par colonne
<code>axis=1</code>		une moyenne par ligne

Si les lignes sont des capteurs, `axis=1` calcule une moyenne par capteur.

Une manière rigoureuse de comprendre `axis` est de considérer que l'axe indiqué est supprimé par le calcul. Avec `axis=0`, les lignes sont agrégées et il reste une valeur par colonne. Avec `axis=1`, les colonnes sont agrégées et il reste une valeur par ligne.

Synthèse de la séance

La chaîne complète de traitement est désormais :

Fichier CSV

Lecture des chaînes de caractères

v

Conversion en listes numériques

v

Création du tableau NumPy

v

Calcul des indicateurs scientifiques

La séance suivante utilisera ces données pour produire des représentations graphiques et réaliser une acquisition.

Exercices — Séance 3 : Exploiter des données scientifiques

Fichiers nécessaires

Copier les fichiers `notes.txt` et `mesures.csv` fournis par l'enseignant dans le dossier `seance3` créé au début du module. Enregistrer également les programmes de cette feuille dans ce dossier. Les chemins simples utilisés ci-dessous fonctionneront alors directement.

Ex. 1 — Module `math` et `random` Utiliser `math` pour calculer racine carrée, puissance, et `random` pour générer 10 mesures aléatoires simulées entre 10 et 20.

Ex. 2 — Lecture d'un fichier texte Lire le fichier `notes.txt` (une valeur par ligne), puis calculer le nombre de notes, leur somme et leur moyenne.

Ex. 3 — Lecture d'un CSV de mesures Charger `mesures.csv` (colonnes `temps` et `temperature`, séparateur `;`) dans deux listes Python, puis calculer la moyenne des températures sans NumPy.

Ex. 4 — Écriture d'un rapport Écrire les résultats de l'Ex. 3 (moyenne, min, max) dans un fichier `rapport.txt`, avec `with open(..., "w")`.

Ex. 5 — Premier tableau NumPy Créer un `ndarray` à partir des températures de l'exercice 3, afficher `shape` et `dtype`, puis convertir les valeurs de °C en K en une seule opération vectorisée.

Ex. 6 — Statistiques et tableau 2D Construire un tableau NumPy 2D (3 capteurs × 5 instants, valeurs de votre choix) et calculer la moyenne par capteur (`axis=1`) et par instant (`axis=0`).

Séance 4

Visualiser et acquérir des mesures

Matplotlib, instrumentation, liaisons série et réseau

Une série de nombres devient plus facile à interpréter lorsqu'elle est représentée graphiquement. Python peut aussi recevoir directement les mesures d'un instrument.

À la fin de la séance, vous devez être capables de produire un graphique scientifique lisible et d'expliquer les étapes d'un dialogue avec un instrument.

Cette séance termine la chaîne commencée avec les fichiers et NumPy. Matplotlib transforme les données en représentations visuelles ; l'instrumentation et la liaison série montrent comment ces données peuvent provenir du monde physique.

Rôle de la visualisation

Un indicateur résume les données ; un graphique montre leur organisation.

Une représentation peut révéler :

- une tendance ou une évolution temporelle ;
- une relation entre deux grandeurs ;
- une dispersion ou une valeur aberrante ;
- un changement brutal ou un dépassement de seuil.

Le graphique doit être choisi en fonction de la question scientifique, pas seulement pour son apparence.

Choisir une représentation

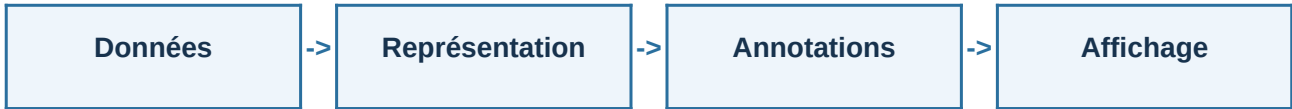
Question	Représentation adaptée
Comment une grandeur évolue-t-elle ?	courbe
Deux variables sont-elles liées ?	nuage de points
Comment les valeurs sont-elles réparties ?	histogramme

Une même série peut être représentée de plusieurs manières, mais chaque graphique met en évidence une information différente.

Relier des points par une courbe suggère un ordre, souvent temporel. Un nuage de points ne suppose pas cette continuité et sert à étudier une relation. Un histogramme regroupe les valeurs par intervalles ; il ne doit pas être confondu avec un diagramme en barres représentant des catégories.

Construire un graphique avec Matplotlib

```
import matplotlib.pyplot as plt temps = [0, 1, 2, 3] temperatures = [20.1, 20.4, 20.9, 21.3] plt.plot(temps, temperatures) plt.xlabel("Temps (s)") plt.ylabel("Température (°C)") plt.title("Évolution de la température") plt.show()
```



Le module `matplotlib.pyplot`, généralement importé sous l'alias `plt`, fournit une interface simple pour construire une figure. Les deux séries transmises à `plot` doivent avoir le même nombre d'éléments : chaque abscisse correspond à une ordonnée.

Représenter une évolution

```
import matplotlib.pyplot as plt temps = [0, 1, 2, 3] temperatures = [20.1, 20.4, 20.9, 21.3] plt.plot( temps, temperatures, color="tab:blue", marker="o", label="Température", )
```

	Argument	Rôle
<code>color</code>		couleur de la courbe
<code>marker</code>		symbole placé sur les mesures
<code>label</code>		nom utilisé dans la légende

Une ligne est pertinente lorsque l'ordre des points possède un sens.

Nuage de points et histogramme

```
import matplotlib.pyplot as plt tensions = [1.0, 2.0, 3.0, 4.0] courants = [0.10, 0.19, 0.31, 0.40] plt.scatter(tensions, courants)
```

Le nuage de points montre la relation entre deux variables mesurées sur les mêmes observations.

```
temperatures = [20.1, 20.4, 20.9, 21.3, 20.8, 20.5] plt.hist(temperatures, bins=10)
```

L'histogramme découpe l'étendue des valeurs en intervalles et compte les observations dans chacun d'eux. Le choix de `bins` influence la lecture de la distribution.

Annoter un graphique scientifique

Un graphique doit pouvoir être compris sans l'explication orale qui l'accompagne.

```
plt.xlabel("Temps (s)") plt.ylabel("Température (°C)") plt.title("Évolution de la température") plt.grid(alpha=0.3) plt.legend()
```

Les axes indiquent la grandeur et son unité. Une légende est nécessaire lorsque plusieurs séries sont tracées ou lorsqu'un `label` doit être identifié.

Un titre décrit la situation représentée sans répéter simplement les axes. Une grille légère facilite la lecture des valeurs mais ne doit pas dominer les données. `legend()` utilise les textes fournis avec l'argument `label` des tracés.

Figure, axes et sous-graphiques

Matplotlib distingue la **figure**, qui contient l'ensemble du document, et les **axes**, qui sont les zones de tracé.

```
import matplotlib.pyplot as plt
temps = [0, 1, 2, 3]
temperatures = [20.1, 20.4, 20.9, 21.3]
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
axes[0].plot(temps, temperatures)
axes[1].hist(temperatures, bins=10)
fig.tight_layout()
```

Les sous-graphiques permettent de comparer plusieurs représentations sans les superposer.

Malgré son nom, un objet `Axes` représente une zone complète de tracé avec ses deux axes gradués. L'interface orientée objet (`fig, axes`) devient plus claire que `plt` dès que la figure contient plusieurs graphiques.

Exporter une figure

Une figure peut être enregistrée pour être intégrée dans un rapport. Cet exemple complet crée la figure avant de l'enregistrer :

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots()
ax.plot([0, 1, 2, 3], [20.1, 20.4, 20.9, 21.3])
ax.set_xlabel("Temps (s)")
ax.set_ylabel("Température (°C)")
fig.savefig("temperature.png", dpi=300, bbox_inches="tight", )
```

`dpi` contrôle la résolution d'une image matricielle ; `bbox_inches="tight"` limite les marges inutiles.

La sauvegarde est généralement effectuée avant `plt.show()`.

Le format PNG convient aux documents courants. Les formats vectoriels comme SVG ou PDF restent nets lors d'un agrandissement. Avec l'interface `pyplot`, la fonction équivalente est `plt.savefig()`.

Chaîne d'instrumentation

Une mesure numérique résulte de plusieurs transformations :

Grandeur physique

v

Capteur

v

Électronique d'acquisition

v

Communication

v

Ordinateur et programme Python

v

Analyse et représentation

Python intervient après la conversion de la grandeur physique en données transmissibles.

Le capteur réagit à une grandeur telle qu'une température ou une tension. L'électronique conditionne et numérise le signal. Le système communique ensuite les valeurs à l'ordinateur, où Python peut automatiser leur stockage, leur analyse et leur représentation.

Liaison matérielle et liaison logicielle

Pour dialoguer avec un instrument, il faut d'abord savoir comment s'y brancher, puis quelles instructions lui envoyer :

Élément	Question associée	Exemples
liaison matérielle	quel câble et quel connecteur ?	USB, série, Ethernet, GPIB
liaison logicielle	quelles règles et commandes ?	SCPI, VISA, Modbus, protocole constructeur

Deux appareils équipés du même connecteur USB ou Ethernet ne parlent pas forcément le même langage.

La liaison matérielle décrit le transport. La liaison logicielle définit la structure, l'ordre et la signification des messages. Avant tout achat ou essai, il faut donc vérifier les deux dans la documentation technique.

Commandes SCPI

SCPI (*Standard Commands for Programmable Instruments*) définit une syntaxe textuelle utilisée par de nombreux instruments de laboratoire.

```
*IDN? MEAS:TEMP? TEMP:UNIT DEG
```

- `*IDN?` demande l'identification de l'instrument ;
- `MEAS:TEMP?` demande une mesure de température ;
- `TEMP:UNIT DEG` règle un paramètre sans attendre nécessairement de réponse ;
- le point d'interrogation signale une requête qui attend une réponse.

Dans tous les cas, seule la documentation de l'instrument fait foi.

SCPI standardise une partie du vocabulaire, mais les commandes réellement prises en charge dépendent de l'instrument. Sa documentation reste la référence. Les commandes sont généralement terminées par un caractère de fin de ligne.

Paramètres d'une liaison série

Une liaison série transmet une suite d'octets. Les deux équipements doivent utiliser des paramètres compatibles.

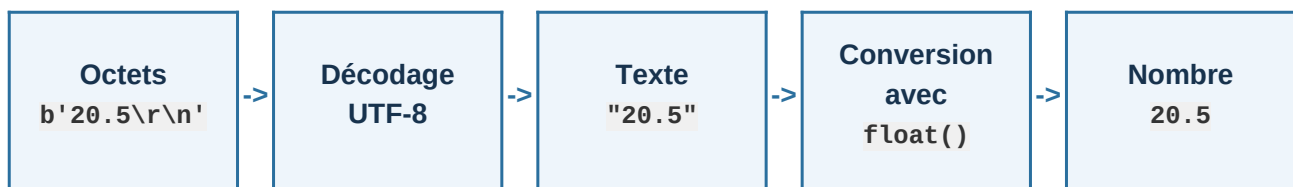
Paramètre	Rôle
port	identifier l'interface, par exemple <code>COM3</code>
baudrate	fixer la vitesse de transmission
format	fixer le nombre de bits, la parité et les bits d'arrêt
timeout	limiter la durée d'attente d'une lecture
fin de ligne	délimiter les messages

Sous Linux, un port peut notamment apparaître sous la forme `/dev/ttyUSB0` ou `/dev/ttyACM0`.

Sous Windows, un port série ne peut généralement être ouvert que par une application à la fois. Le nom d'un adaptateur USB-série peut aussi changer après un débranchement.

Octets, texte et nombre

Le port série échange des `bytes`, alors que le programme veut généralement manipuler du texte puis un nombre.



```
ligne = b"20.5\r\n" # exemple de réponse reçue
texte = ligne.decode("utf-8").strip()
mesure = float(texte)
```

L'encodage doit être identique du côté de l'émetteur et du récepteur.

Un octet est une valeur numérique comprise entre 0 et 255. Un encodage comme UTF-8 définit comment une suite d'octets représente des caractères. `strip()` retire ici les caractères de fin de ligne avant la conversion numérique.

Lire avec pyserial

```
import serial with serial.Serial("COM3", baudrate=19200, timeout=1) as port: ligne = port.readline() texte = ligne.decode("utf-8").strip() mesure = float(texte)
```

Le bloc `with` ferme automatiquement le port. `readline()` lit jusqu'à une fin de ligne ou jusqu'à l'expiration du délai.

L'ouverture peut échouer si le port n'existe pas, est déjà utilisé ou si les droits sont insuffisants. Une lecture peut être vide après un timeout et une réponse peut être invalide. Une application réelle doit donc vérifier les données reçues avant leur conversion.

Envoyer une commande

Une commande textuelle doit être encodée avant son envoi :

```
import serial with serial.Serial("COM3", baudrate=19200, timeout=1) as port: commande = "*IDN?\n".encode("utf-8") port.write(commande) reponse = port.readline().decode("utf-8").strip()
```

Commande textuelle

v

Encodage en octets

v

Envoi et réception

v

Décodage des octets

v

Réponse textuelle

La terminaison `\n` est ici incluse dans la commande car de nombreux instruments attendent une fin de ligne avant de traiter le message. La terminaison exacte doit être vérifiée dans la documentation de l'appareil.

Communiquer par le réseau

Un instrument Ethernet est généralement identifié par deux informations :

Information	Rôle	Exemple
adresse IP	identifier l'instrument sur le réseau	192.168.0.26
port TCP	identifier le service qui reçoit les commandes	8000

Le dialogue suit ensuite les mêmes étapes qu'avec une liaison série :



Les valeurs à utiliser et le protocole de dialogue sont indiqués dans la documentation de l'instrument. Une prise Ethernet ne garantit pas à elle seule que l'appareil accepte des commandes SCPI par une connexion TCP directe.

Python fournit le module `socket` pour les communications réseau de bas niveau. Les constantes et méthodes nécessaires dépendent du protocole retenu ; elles ne sont pas à mémoriser dans ce cours. En pratique, une bibliothèque constructeur ou VISA peut fournir une interface plus simple. On commencera donc toujours par la documentation de l'instrument plutôt que par un exemple de `socket` générique.

Acquisition avec une carte Arduino

Une carte Arduino peut envoyer périodiquement une mesure terminée par un retour à la ligne. Python peut alors :

1. ouvrir la liaison série ;
2. lire et convertir chaque mesure ;
3. stocker les valeurs ;
4. calculer des indicateurs avec NumPy ;
5. tracer et sauvegarder un graphique.



v

Python

v

NumPy

v

Matplotlib

Une source simulée permet de tester le programme lorsqu'aucun matériel n'est disponible.

Synthèse du cours

Les quatre séances construisent une chaîne complète :

Étape

programmer
stocker
calculer
visualiser
acquérir

Outils principaux

variables, conditions, fonctions, boucles
listes, fichiers, CSV
NumPy et statistiques descriptives
Matplotlib
instrument, liaisons série ou réseau, Arduino

L'objectif est de transformer une grandeur physique en données interprétables, puis en résultat scientifique communicable.

Chaque étape possède une responsabilité distincte. Cette séparation permet de tester les traitements avec des données simulées avant de connecter le matériel, puis de remplacer la simulation par une acquisition réelle sans reconstruire tout le programme.

Exercices — Séance 4 : Visualiser et acquérir des mesures

Fichier nécessaire

Copier `mesures.csv` depuis le dossier `seance3` vers le dossier `seance4`, puis enregistrer les programmes de cette feuille dans `seance4`.

Ex. 1 — Première courbe À partir du CSV de la séance 3, tracer température en fonction du temps avec `plt.plot`, en ajoutant titre et légendes des axes.

Ex. 2 — Nuage de points et histogramme Dans un même script, créer d'abord une figure contenant un nuage de points (deux séries de mesures liées), puis une seconde figure contenant l'histogramme d'une série de valeurs.

Ex. 3 — Personnalisation et export Reprendre l'Ex. 1 : changer couleur/style de ligne, ajouter une grille et une légende, puis exporter le graphique en `.png`.

Ex. 4 — Détection de dépassement de seuil Ajouter au graphique une ligne horizontale représentant un seuil (`plt.axhline`), et faire apparaître en rouge les points qui le dépassent.

Ex. 5 — Sous-figures Créer une figure avec 2 sous-graphiques côte à côte (courbe + histogramme) pour la même série de mesures, avec `plt.subplots`.

Ex. 6 — Lecture d'un port série (si matériel disponible) Avec `pyserial`, lister les ports avec `serial.tools.list_ports.comports()`, se connecter à une carte, lire 10 mesures, les stocker dans une liste puis les tracer. Sans matériel, relire progressivement les lignes de `data/donnees/mesures.csv` comme si elles arrivaient du port série.